

Figure 2: Sample enterprise search chatbot screen

the requirements, the degree of customisation needed, the resources available for development and maintenance of custom solutions, and the cost comparison between managed services and proprietary solutions.

Regardless of the choice of implementation, the integration of RAG and LLMs along with the power of genAI is significantly transforming the enterprise search and

question-answering (QA) systems and is seeing widespread adoption in enterprises. **END** 

By: Lokesh Chenta

The author works as a senior principal software architect at Sabre, India. He is experienced in the design and development of enterprise products and technology platforms that perform at high scale.

■■ Continued from page...75

Payload optimisation created lightweight serialisers using only six fields instead of thirty, enabled GZIP compression via NGINX, and implemented field-level API selection.

Infrastructure scaling added two read replicas for the product database, deployed an NGINX load balancer distributing traffic across four application servers, and set up Prometheus and Grafana monitoring.

Results were dramatic. Average response time decreased from 10 seconds to 250 milliseconds—a 96% improvement. Database queries per request dropped from 150 to 8. Server CPU utilisation at peak load reduced from 95% to 45%. The platform successfully handled 200,000 concurrent users during flash sales with zero downtime over six months.

Cache hit ratio reached 85%, meaning 85% of requests were served from Redis without touching the database. Connection pooling prevented connection exhaustion during traffic spikes. Background task processing eliminated user-facing delays for non-critical operations.

Putting it all together

Reducing request overhead in Django microservices requires combining multiple strategies. Start with measurement using Django Debug Toolbar and Prometheus to identify bottlenecks. Fix N+1 queries with `select_related` and `prefetch_related`. Add strategic indexes. Implement Redis caching to reduce database load by 80%. Offload time-consuming tasks to Celery. Enable GZIP compression and NGINX load balancing.

The Django ecosystem, combined with PostgreSQL, Redis, Celery, and NGINX, provides battle-tested tools for building high-performance microservices on Linux. Build performance from the start—design for caching, optimise queries, and plan for scale before you need it. These practices prevent problems rather than fixing them after they impact users. **END** 

References

- Django Official Documentation, <https://docs.djangoproject.com/>
- PostgreSQL Documentation, <https://www.postgresql.org/docs/>
- Redis Documentation, <https://redis.io/documentation>
- Celery Distributed Task Queue, <https://docs.celeryproject.org/>
- Django REST Framework, <https://www.django-rest-framework.org/>
- NGINX Official Documentation, <https://nginx.org/en/docs/>
- Django Channels Documentation, <https://channels.readthedocs.io/>
- Prometheus Monitoring System, <https://prometheus.io/docs/>
- Grafana Visualization Platform, <https://grafana.com/docs/>
- Kong API Gateway Documentation, <https://docs.konghq.com/>
- PgBouncer Connection Pooler, <https://www.pgbouncer.org/>
- Unicorn WSGI HTTP Server, <https://docs.gunicorn.org/>

By: Naser Ghayem

The author is a senior backend developer specialising in Python and distributed systems. He enjoys contributing to open source projects and mountain biking.